

Part 1:

1. My proposed row key is hash_prefix#city#make#vin. Bigtable stores rows in lexicographic order and partitions contiguous key ranges into tablets, so row key design determines data distribution. Adding a hash prefix spreads writes across tablets and avoids hotspotting when many inserts share similar prefixes. The tradeoff is that queries by City or Make are less efficient because rows are no longer stored contiguously and must be scanned across multiple hash buckets.
2. I would make the column families: id, registration, vehicle, specs, and utility. Column families act as locality groups in Bigtable, meaning related columns are stored together on disk. This allows frequently accessed attributes, like city, make, and model, to be read without scanning larger or less-used data. Thus, improving read efficiency and reducing I/O during distributed scans.
3. A hashed shard key provides even data distribution and avoids hotspotting, which improves write scalability across tablets. However, it reduces query efficiency because related rows are spread across partitions. A range-based shard key keeps related rows together, making range scans more efficient but can create hotspots if there are several writes targeting the same key range.

Part 4:

Average /insert-fast latency: 0.005781435966491699

Average /insert-safe latency: 0.023543167114257812

The /insert-fast endpoint is significantly faster because it uses a write concern of $w=1$, meaning the write is only acknowledged by the primary node. In contrast, /insert-safe uses $w=\text{"majority"}$ which requires acknowledgement from multiple replica set members before returning success, increasing latency. Based on the CAP theorem, I would choose the faster primary-only write in a scenario where low latency is more important than durability, such as logging a user's behavior on a webpage and tracking social media likes. In such cases, losing a small amount of recent data during a failure is acceptable, and the performance benefits of faster writes improve overall system responsiveness.